

پروژه اول درس معماری کامپیوتر

طراحی و پیاده‌سازی پردازنده Single-Cycle با Verilog

استاد: دکتر دهیادگاری

دستیار آموزشی: آقای دوستی

نام دانشجو: مبینا پورافشار

نیمسال تحصیلی: 4042

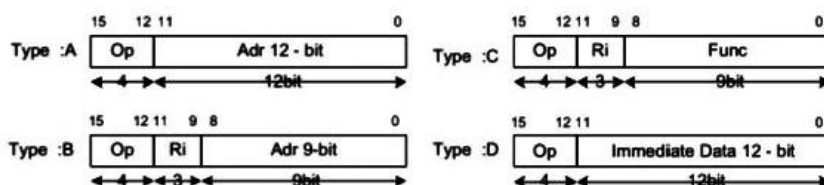
1. مقدمه و صورت پروژه

هدف این پروژه، طراحی و پیاده‌سازی یک پردازنده Single-Cycle شانزده‌بیتی با زبان Verilog و بررسی صحت عملکرد آن با استفاده از تست‌بنچ‌های مستقل و یک تست‌بنچ یکپارچه است. در این معماری، هر دستورالعمل در یک سیکل کلاک کامل اجرا می‌شود؛ بنابراین واکنشی دستور، تولید سیگنال‌های کنترلی، اجرای عملیات، دسترسی به حافظه و بازنویسی نتیجه همگی باید در همان سیکل انجام شوند.

پردازنده طراحی شده دارای 8 رجیستر 16 بیتی، حافظه دستور و داده با ظرفیت $4K \times 16$ و یک ISA سفارشی شامل دستورات حافظه‌ای، رجیستری، فوری، انشعب و پرش است. در این طراحی، R0 نقش Accumulator را دارد و مبنای اصلی بسیاری از عملیات محاسباتی و منطقی است.

پردازنده‌ای با مشخصات زیر در نظر بگیرید:

- رجیستر فایل شامل 8 رجیستر 16 بیتی (R0 تا R7) است، که R0 به عنوان Accumulator مورد استفاده قرار می‌گیرد.
- توانایی آدرس‌دهی حافظه‌ای به ظرفیت $4K \times 16\text{bit}$
- چهار نوع دستور با قالب‌های زیر:



2. مشخصات معماری و ISA

مشخصات اصلی پردازنده به صورت زیر است:

ویژگی	مقدار
نوع معماری	Single-Cycle
طول داده	16 بیت
طول آدرس	12 بیت
تعداد رجیسترها	8 عدد
رجیستر ویژه	R0 به عنوان Accumulator
حافظه دستور	4096 x 16-bit
حافظه داده	4096 x 16-bit
زبان پیاده سازی	Verilog HDL

این معماری به گونه ای طراحی شده است که یکی از مسیرهای خواندن RegisterFile به صورت ثابت به رجیستر R0 متصل باشد. این تصمیم مستقیماً با نقش accumulator برای R0 سازگار است و باعث ساده تر شدن دیتا پث و کنترل شده است.

3. فرمت دستورها و مجموعه دستورات عمل ها

پردازنده از چهار نوع فرمت دستور پشتیبانی می کند:

نوع	ساختار	کاربرد
A	Op[15:12], Address[11:0]	Load, Store, Jump
B	Op[15:12], Ri[11:9], Addr[8:0]	BranchZ
C	Op[15:12], Ri[11:9], Funct[8:0]	دستورات رجیستری
D	Op[15:12], Imm[11:0]	دستورات فوری

جدول مجموعه دستورالعمل‌ها:

Mnemonic		Description	Opcode	Function	Type
Load	adr-12	$R0 \leftarrow M[\text{adr-12}]$	0000	-	A
Store	adr-12	$M[\text{adr-12}] \leftarrow R0$	0001	-	A
Jump	adr-12	$PC \leftarrow \text{adr-12}$	0010	-	A
BranchZ	Ri, adr-9	If $(R0=Ri)$ $PC[8:0] \leftarrow \text{adr-9}$	0100	-	B
MoveTo	Ri	$Ri \leftarrow R0$	1000	000000001	C
MoveFrom	Ri	$R0 \leftarrow Ri$	1000	000000010	C
Add	Ri	$R0 \leftarrow R0 + Ri$	1000	000000100	C
Sub	Ri	$R0 \leftarrow R0 - Ri$	1000	000001000	C
And	Ri	$R0 \leftarrow R0 \text{ AND } Ri$	1000	000010000	C
Or	Ri	$R0 \leftarrow R0 \text{ OR } Ri$	1000	000100000	C
Not	Ri	$R0 \leftarrow \text{NOT } Ri$	1000	001000000	C
Nop	-	No Operation	1000	010000000	C
Addi	Imm-12	$R0 \leftarrow R0 + \text{Imm-12}$	1100	-	D
Subi	Imm-12	$R0 \leftarrow R0 - \text{Imm-12}$	1101	-	D
Andi	Imm-12	$R0 \leftarrow R0 \text{ AND } \text{Imm-12}$	1110	-	D
Ori	Imm-12	$R0 \leftarrow R0 \text{ OR } \text{Imm-12}$	1111	-	D

• جدول صحت ALU به صورت زیر است.

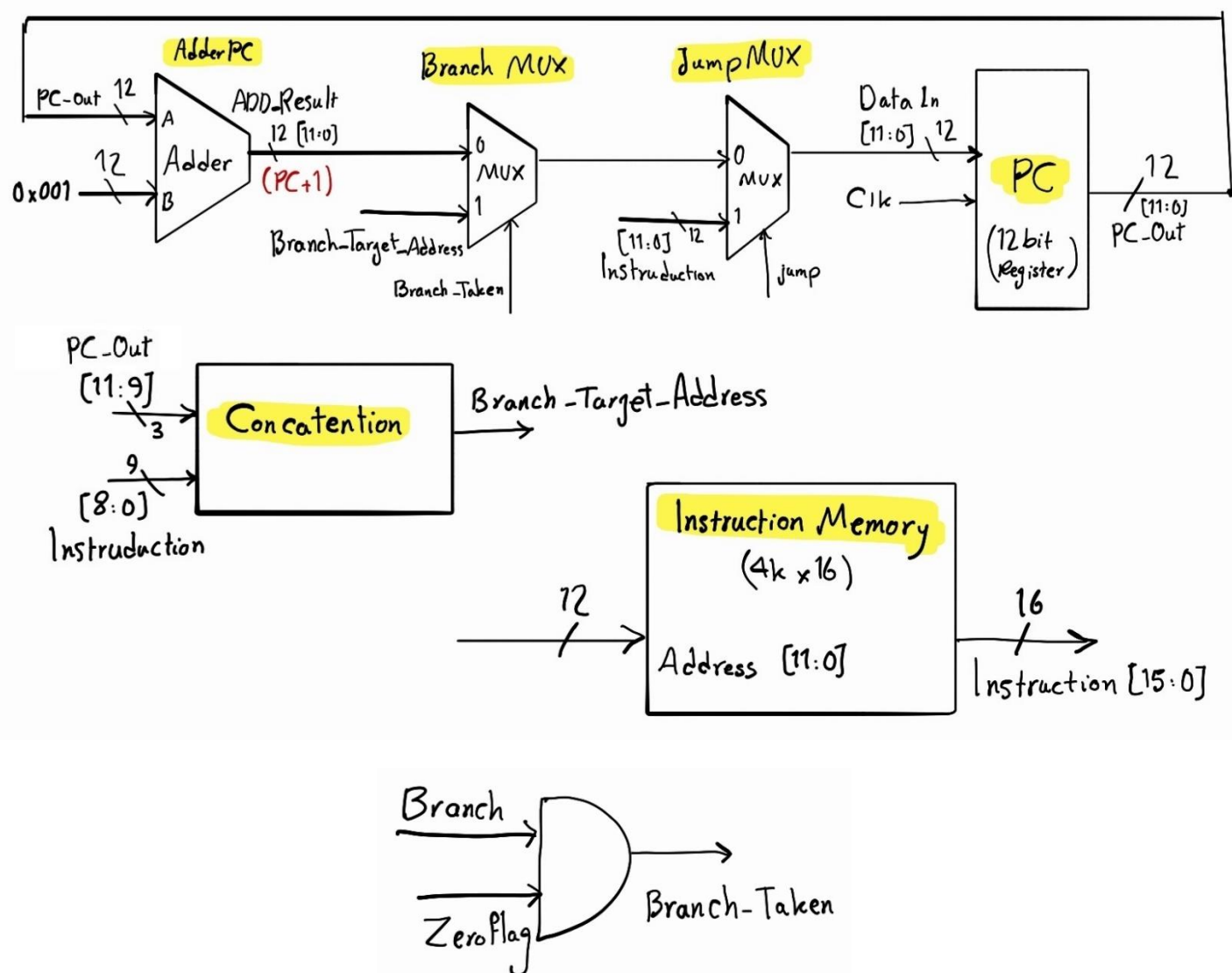
Operation	y
000	$In1 + In2$
001	$In1 - In2$
010	$In1 \& In2$
011	$In1 In2$
100	Not In1
101	In1
110	In2

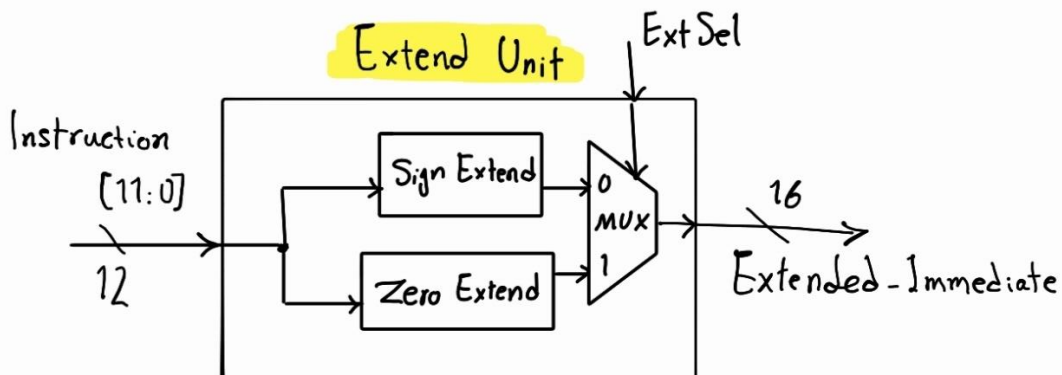
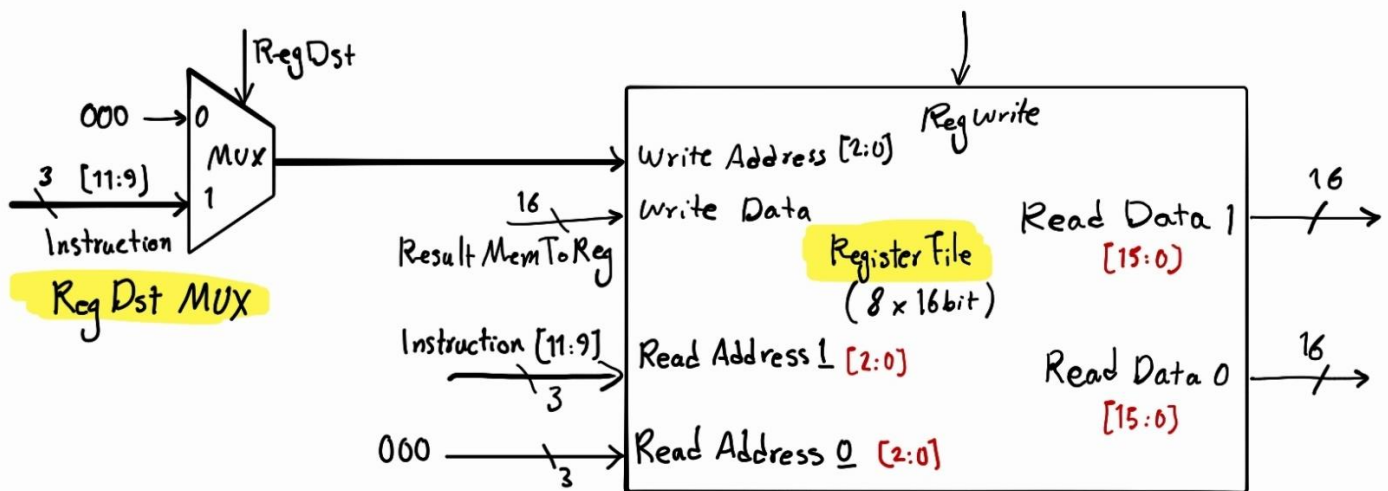
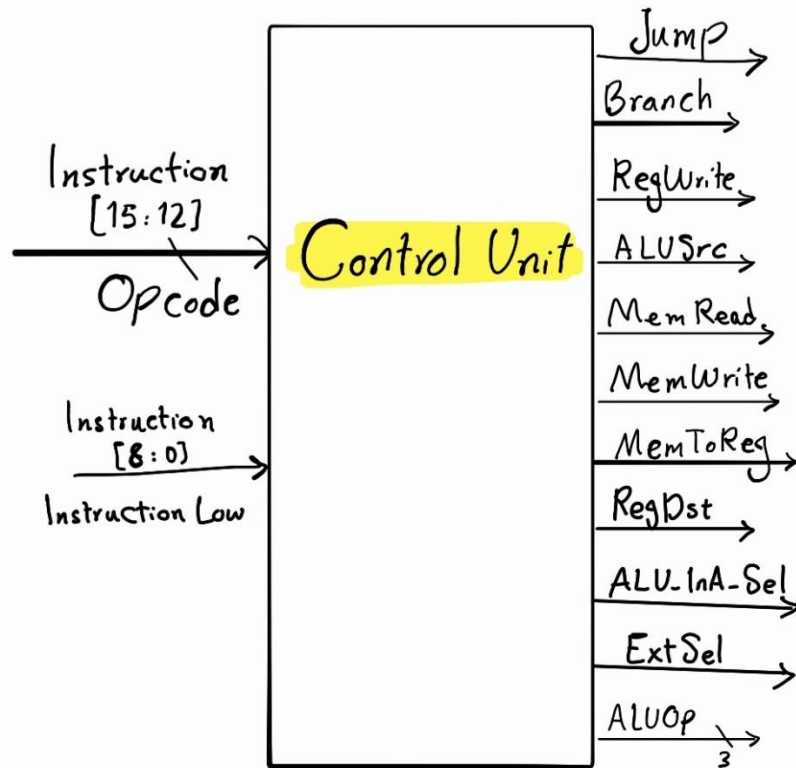
$In2, In1$ و Operation ورودی‌های ALU و y خروجی ALU است.

4. معماری دیتاپث

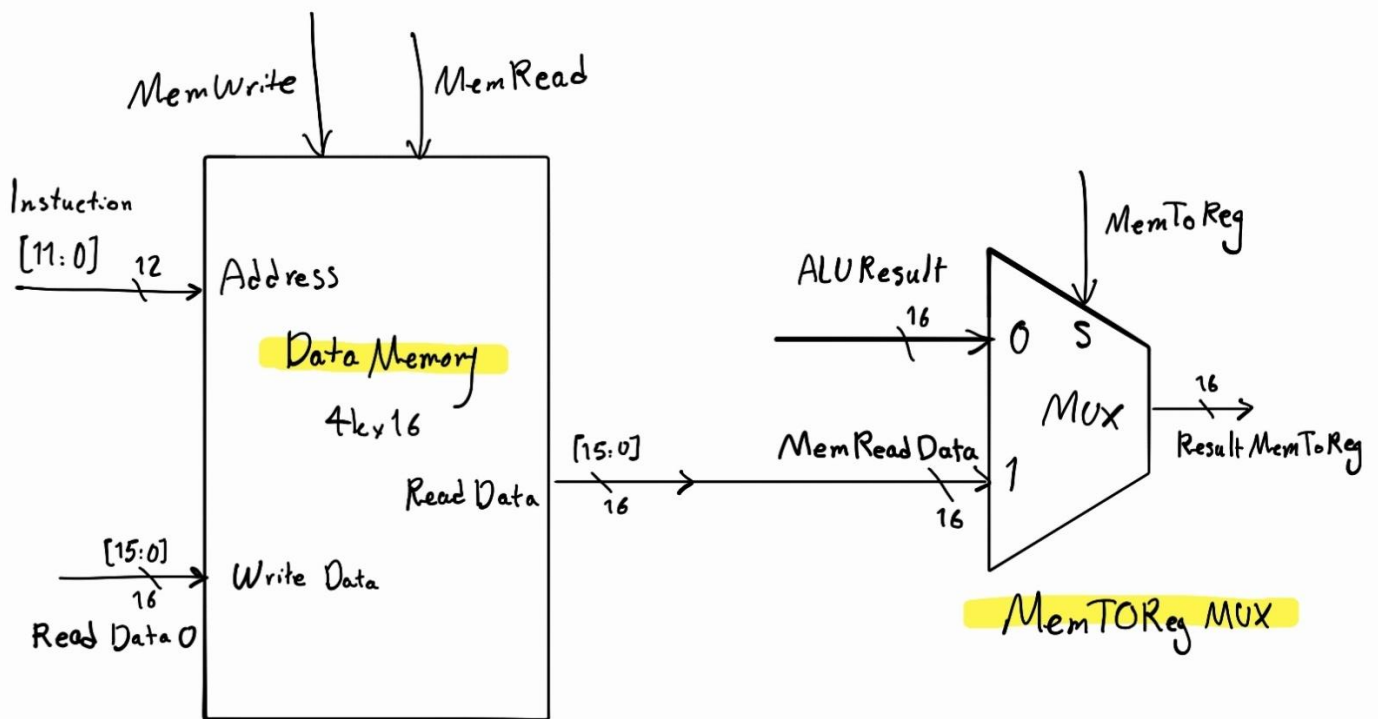
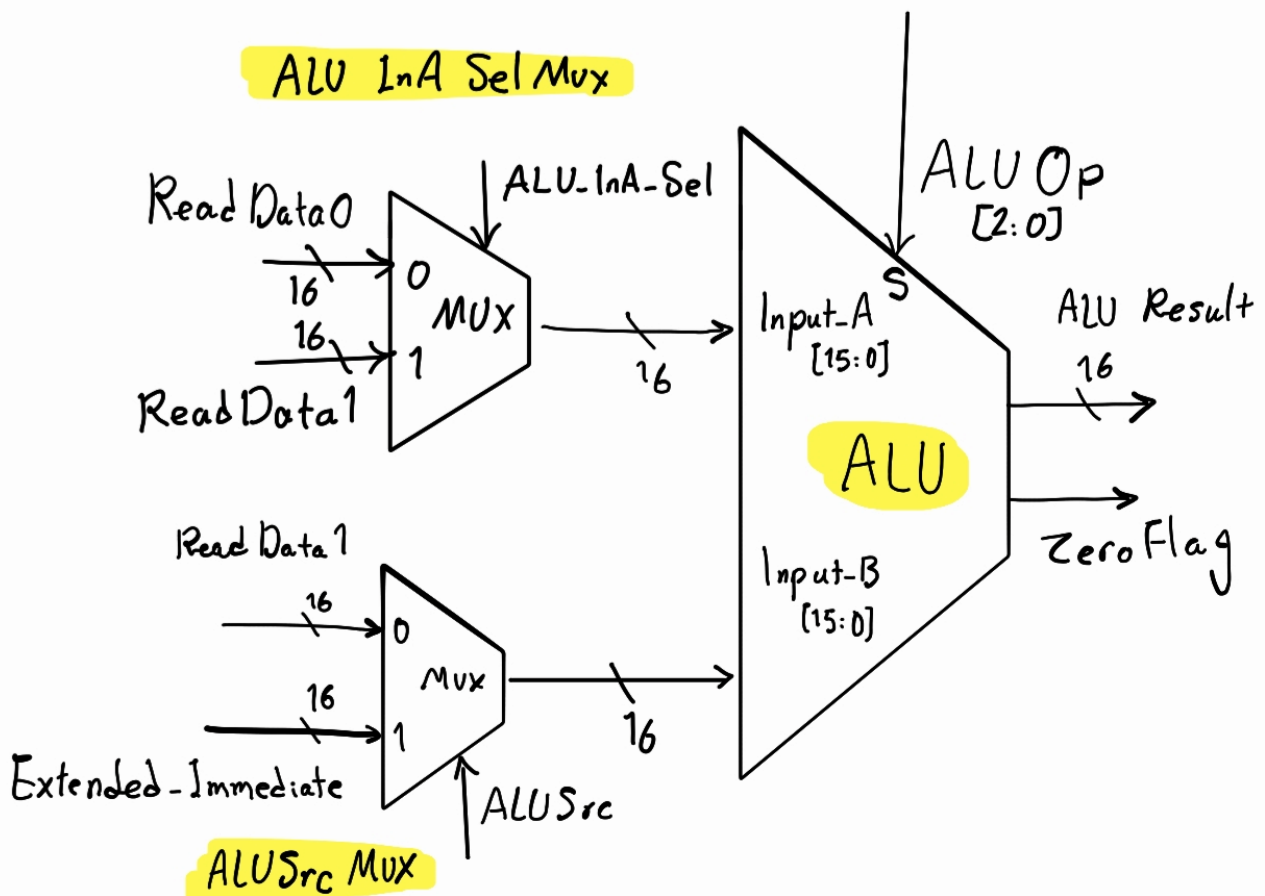
دیتاپث پردازنده شامل ماژول‌های PC، AdderPC، InstructionMemory، ControlUnit، RegisterFile، ExtendUnit، ALU و DataMemory است. علاوه بر این‌ها، چند مالتی‌پلکسر منطقی نیز برای انتخاب مسیر داده در نظر گرفته شده‌اند؛ از جمله $MemToReg$ ، ALU_InA_Sel ، $ALUSrc$ ، $RegDst$ ، مسیر $branch$ و مسیر $jump$.

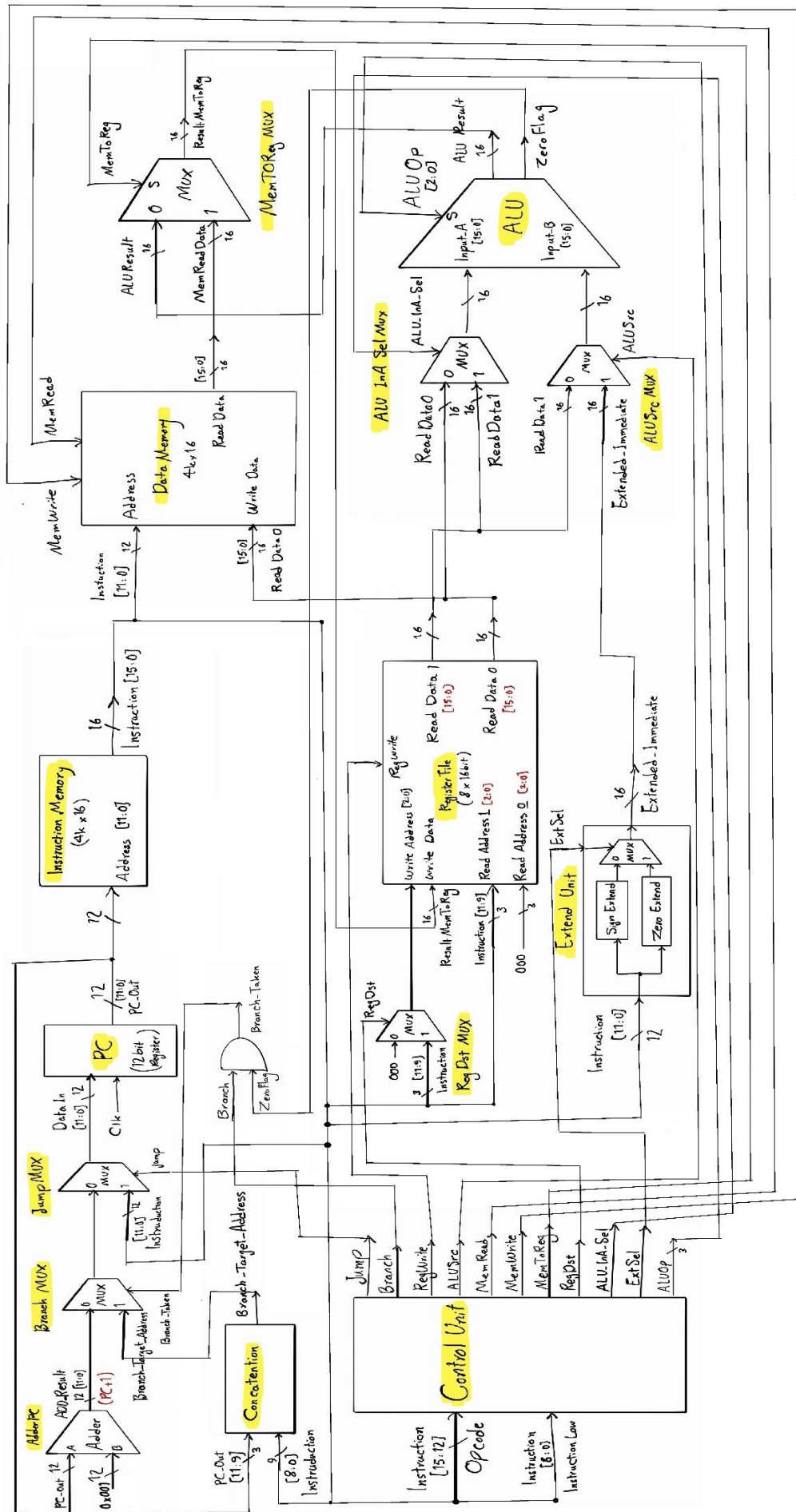
نکته مهم این است که این MUXها در کد Verilog به صورت ماژول جداگانه instantiate نشده‌اند، بلکه با منطق ترکیبی و دستورات assign در ماژول SingleCycleProcessor پیاده‌سازی شده‌اند. این موضوع از نظر معماری کاملاً معتبر است و فقط شیوه نمایش در کد را ساده‌تر کرده است.





ALU InA Sel Mux





5. شرح ماژول‌ها

در این پروژه، هر بخش به صورت یک ماژول مستقل پیاده‌سازی شده است.

- PC: یک ثابت 12 بیتی است که در لبه بالارونده کلاک مقدار next_pc را دریافت می‌کند و با reset آسنکرون به صفر برمی‌گردد.
- AdderPC: مقدار $PC + 1$ را تولید می‌کند و برای اجرای ترتیبی برنامه استفاده می‌شود.
- InstructionMemory: حافظه دستور 4096 x 16 است. این حافظه ابتدا با NOP مقداردهی می‌شود و سپس فایل program.hex را با readmemh بارگذاری می‌کند. چون این فایل در پوشه tb / قرار دارد، شبیه‌سازی باید از داخل همان پوشه اجرا شود تا بارگذاری بدون خطا انجام شود.
- ControlUnit: بر اساس opcode و در صورت نیاز funct سیگنال‌های کنترلی را تولید می‌کند.
- RegisterFile: شامل 8 رجیستر 16 بیتی است. خواندن آن ترکیبی و نوشتن آن سنکرون است. در این طراحی، read_addr_0 عملاً روی R0 قرار می‌گیرد تا نقش accumulator حفظ شود.
- ExtendUnit: مقدار immediate دوازده‌بیتی را بسته به ExtSel به صورت sign-extension یا zero-extension به 16 بیت تبدیل می‌کند.
- ALU: عملیات حسابی و منطقی را انجام می‌دهد و سیگنال zero_flag را برای branch تولید می‌کند.
- DataMemory: برای Load و Store به کار می‌رود؛ خواندن آن ترکیبی و نوشتن آن سنکرون است.
- SingleCycleProcessor: ماژول top-level است که همه زیرماژول‌ها را به هم متصل می‌کند و کل رفتار پردازنده را می‌سازد.

6. تحلیل واحد کنترل و مسیر PC

واحد کنترل سیگنال‌های اصلی زیر را تولید می‌کند:

Jump, Branch, RegWrite, ALUSrc, MemRead, MemWrite, MemToReg, RegDst, ALU_InA_Sel, ExtSel, ALUOp

نمونه رفتار کنترلی:

- Load: خواندن از حافظه و نوشتن نتیجه در R0
- Store: نوشتن مقدار R0 در حافظه
- BranchZ: انجام تفریق $R0 - Ri$ و بررسی zero_flag

- MoveTo: انتقال از R0 به Ri

- MoveFrom: انتقال از Ri به R0

- Ori و Andi: استفاده از zero-extension

- Addi و Subi: استفاده از sign-extension

برای به‌روزرسانی PC نیز ابتدا pc_plus_1 ساخته می‌شود، سپس در صورت برقرار بودن شرط branch مقصد شاخه انتخاب می‌شود و در نهایت اگر دستور Jump باشد، آدرس پرش اولویت نهایی را دارد. در کد این منطق به‌صورت زیر پیاده‌سازی شده است:

- $\text{branch_target} = \{\text{pc_out}[11:9], \text{instruction}[8:0]\}$

- $\text{branch_taken} = \text{Branch} \ \& \ \text{zero_flag}$

- $\text{next_pc} = \text{Jump} ? \text{jump_target} : \text{branch_or_next_pc}$

7. روش شبیه‌سازی در ModelSim

در این پروژه، فایل‌های طراحی از پوشه src/ و تست‌بنچ‌ها از پوشه tb/ کامپایل شده‌اند. همچنین شبیه‌سازی از داخل پوشه tb/ اجرا شده است تا ماژول InstructionMemory بتواند فایل program.hex را با دستور readmemh("program.hex", mem) به‌درستی بارگذاری کند.

Name	Status	Type	Order
RegisterFile.v	✓	Verilog	1
ExtendUnit_tb.v	✓	Verilog	11
ControlUnit_tb.v	✓	Verilog	16
ExtendUnit.v	✓	Verilog	2
RegisterFile_tb.v	✓	Verilog	12
DataMemory.v	✓	Verilog	4
InstructionMemory.v	✓	Verilog	3
PC.v	✓	Verilog	6
AdderPC.v	✓	Verilog	7
SingleCycleProcessor_tb.v	✓	Verilog	9
ALU_tb.v	✓	Verilog	10
InstructionMemory_tb.v	✓	Verilog	17
AdderPC_tb.v	✓	Verilog	15
PC_tb.v	✓	Verilog	14
DataMemory_tb.v	✓	Verilog	13
ControlUnit.v	✓	Verilog	5
ALU.v	✓	Verilog	0
SingleCycleProcessor.v	✓	Verilog	8

8. تحلیل برنامه program.hex

برنامه نمونه اکثر قابلیت‌های ISA را پوشش می‌دهد: عملیات فوری، انتقال رجیستری، دستورات منطقی، branch، load/store و jump.

جدول Trace اجرای برنامه

جدول زیر رفتار برنامه را دستور به دستور نشان می‌دهد:

Step	PC	Hex	دستور	عملیات	نتیجه
1	000	C005	Addi 0x005	$R0 \leftarrow R0 + 0005$	R0=0005
2	001	8201	MoveTo R1	$R1 \leftarrow R0$	R1=0005
3	002	C003	Addi 0x003	$R0 \leftarrow R0 + 0003$	R0=0008
4	003	8401	MoveTo R2	$R2 \leftarrow R0$	R2=0008
5	004	8202	MoveFrom R1	$R0 \leftarrow R1$	R0=0005
6	005	8404	Add R2	$R0 \leftarrow R0 + R2$	R0=000D
7	006	8208	Sub R1	$R0 \leftarrow R0 - R1$	R0=0008
8	007	8601	MoveTo R3	$R3 \leftarrow R0$	R3=0008
9	008	420C	BranchZ R1, 00C	اگر $R0 - R1 = 0$	شاخه گرفته نمی‌شود
10	009	8210	And R1	$R0 \leftarrow R0 \text{ AND } R1$	R0=0000
11	00A	480C	BranchZ R4, 00C	اگر $R0 - R4 = 0$	شاخه گرفته می‌شود، PC=00C
12	00C	8220	Or R1	$R0 \leftarrow R0 \text{ OR } R1$	R0=0005
13	00D	8240	Not R1	$R0 \leftarrow \text{NOT } R1$	R0=FFFA
14	00E	E00F	Andi 0x00F	$R0 \leftarrow R0 \text{ AND } 000F$	R0=000A
15	00F	F0F0	Ori 0x0F0	$R0 \leftarrow R0 \text{ OR } 00F0$	R0=00FA
16	010	D00F	Subi 0x00F	$R0 \leftarrow R0 - 000F$	R0=00EB

17	011	1020	Store 0x020	$\text{MEM}[020] \leftarrow R0$	$\text{MEM}[020]=00\text{EB}$
18	012	CFFF	Addi 0xFFFF	$R0 \leftarrow R0 - 1$	$R0=00\text{EA}$
19	013	0020	Load 0x020	$R0 \leftarrow \text{MEM}[020]$	$R0=00\text{EB}$
20	014	8A01	MoveTo R5	$R5 \leftarrow R0$	$R5=00\text{EB}$
21	015	4618	BranchZ R3, 018	اگر $R0 - R3 = 0$	شاخه گرفته نمی شود
22	016	4A19	BranchZ R5, 019	اگر $R0 - R5 = 0$	شاخه گرفته می شود، $\text{PC}=019$
23	019	8080	Nop	بدون عملیات	بدون تغییر
24	01A	201C	Jump 0x01C	$\text{PC} \leftarrow 01\text{C}$	ورود به پایان برنامه
25	01C	201C	Jump 0x01C	$\text{PC} \leftarrow 01\text{C}$	self-loop
26	01C	201C	Jump 0x01C	$\text{PC} \leftarrow 01\text{C}$	پایدار
27	01C	201C	Jump 0x01C	$\text{PC} \leftarrow 01\text{C}$	پایدار
28	01C	201C	Jump 0x01C	$\text{PC} \leftarrow 01\text{C}$	پایدار
29	01C	201C	Jump 0x01C	$\text{PC} \leftarrow 01\text{C}$	پایدار
30	01C	201C	Jump 0x01C	$\text{PC} \leftarrow 01\text{C}$	پایدار

Truth Table کنترول یونیت - بر اساس opcode

[illegible]

سطر «Reg-Type» فقط نشان می‌دهد که این opcode وارد case داخلی funct می‌شود؛ مقادیر * در جدول بعدی تعیین شده‌اند.

Truth Table کنترل یونیت - بر اساس funct در نوع C (سیگنال‌های کنترل برای opcode = 1000

[illegible]

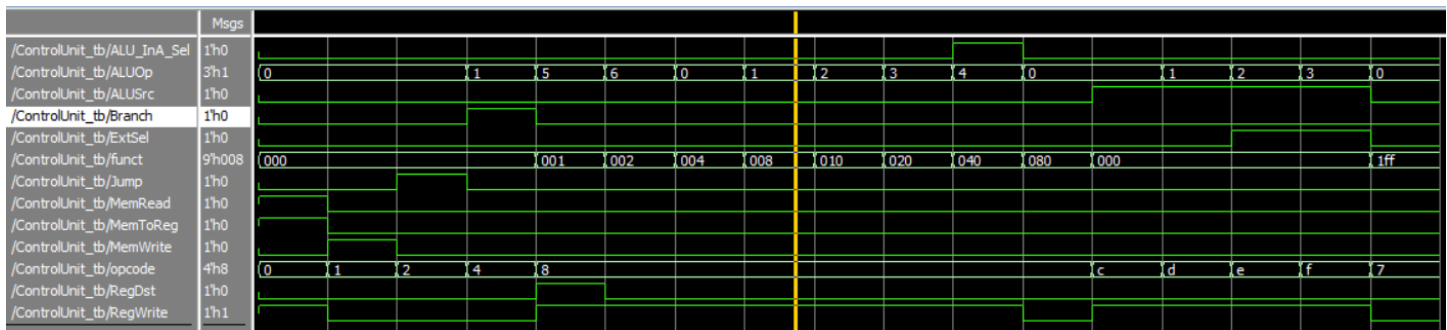
9. تحلیل تست‌بنچ‌ها و Waveform‌ها

برای بررسی صحت عملکرد طراحی، ابتدا هر ماژول به صورت مستقل شبیه‌سازی شده است. در هر تست‌بنچ، ورودی‌های مختلف به ماژول اعمال شده و خروجی‌ها در waveform بررسی شده‌اند. همچنین transcript خروجی ModelSim نیز نمایش داده شده است که نشان می‌دهد مقادیر واقعی با مقادیر مورد انتظار تست‌بنچ مقایسه شده‌اند.

9-1. تست واحد کنترل ControlUnit_tb

هدف این تست، بررسی عملکرد واحد کنترل در تولید صحیح سیگنال‌های کنترلی برای دستورهای مختلف پردازنده است. واحد کنترل بر اساس مقدار opcode و در بعضی دستورها بر اساس مقدار funct تعیین می‌کند که کدام مسیرهای داده فعال شوند، ALU چه عملیاتی انجام دهد، آیا دسترسی به حافظه لازم است یا نه، و مقدار بعدی PC چگونه انتخاب شود.

در این تست، حالت‌های مختلف opcode شامل دستورهای حافظه‌ای، پرش، انشعاب، دستورهای رجیستری و دستورهای immediate بررسی شده‌اند. همچنین برای opcode مربوط به دستورهای نوع C، مقادیر مختلف funct اعمال شده‌اند تا عملکردهای متفاوت ALU و مسیرهای انتقال داده بررسی شوند.



تحلیل Waveform

در waveform مربوط به ControlUnit_tb، تغییرات opcode و funct باعث تغییر سیگنال‌های کنترلی شده است. برای دستور Load، سیگنال‌های مربوط به خواندن از حافظه و نوشتن در رجیستر فعال می‌شوند؛ یعنی MemRead، MemToReg و RegWrite باید مقدار فعال داشته باشند. این رفتار نشان می‌دهد که مسیر داده از حافظه داده به سمت فایل ثبات‌ها به درستی توسط واحد کنترل انتخاب می‌شود.

در دستور Store، برخلاف Load، نیازی به نوشتن در رجیستر وجود ندارد و سیگنال اصلی فعال شده MemWrite است. این موضوع در waveform قابل مشاهده است و نشان می‌دهد که واحد کنترل بین دستورهای خواندن از حافظه و نوشتن در حافظه تمایز درستی قائل شده است.

برای دستور Jump، سیگنال Jump فعال می‌شود تا مسیر بعدی PC از حالت عادی $PC + 1$ خارج شده و آدرس پرش انتخاب شود. در دستور BranchZ نیز سیگنال Branch فعال می‌شود و ALUOp به گونه‌ای تنظیم می‌گردد که عملیات تفریق انجام شود؛ زیرا شرط branch بر اساس مقایسه R0 و Ri و فعال شدن zero_flag تعیین می‌شود.

در بخش مربوط به opcode دستورهای رجیستری، مقدار funct تغییر می‌کند و به دنبال آن مقدار ALUOp نیز تغییر می‌کند. این قسمت نشان می‌دهد که واحد کنترل می‌تواند دستورهای مثل جمع، تفریق، عملیات منطقی، انتقال داده و NOP را از روی فیلد funct تشخیص دهد.

همچنین در opcode‌های مربوط به دستورهای immediate، سیگنال ALUSrc فعال می‌شود تا ورودی دوم ALU از مقدار immediate گرفته شود. برای دستورهای منطقی immediate مانند ANDi و Ori، سیگنال ExtSel حالت zero-extension را انتخاب می‌کند، در حالی که برای دستورهای حسابی immediate مانند Addi و Subi از sign-extension استفاده می‌شود. این رفتار نشان می‌دهد که واحد کنترل با نوع دستور و نحوه تفسیر immediate هماهنگ است.

تحلیل Transcript

```
# ===== ControlUnit_tb =====
# OK: opcode=0000 funct=000000000
# OK: opcode=0001 funct=000000000
# OK: opcode=0010 funct=000000000
# OK: opcode=0100 funct=000000000
# OK: opcode=1000 funct=000000001
# OK: opcode=1000 funct=000000010
# OK: opcode=1000 funct=000000100
# OK: opcode=1000 funct=000001000
# OK: opcode=1000 funct=000010000
# OK: opcode=1000 funct=000100000
# OK: opcode=1000 funct=001000000
# OK: opcode=1000 funct=010000000
# OK: opcode=1100 funct=000000000
# OK: opcode=1101 funct=000000000
# OK: opcode=1110 funct=000000000
# OK: opcode=1111 funct=000000000
# OK: opcode=0111 funct=111111111
# ControlUnit_tb PASSED
```

در transcript این تست، برای هر ترکیب از opcode و funct عبارت OK چاپ شده است. این یعنی تست‌بنچ خروجی‌های واحد کنترل را با مقادیر مورد انتظار مقایسه کرده و در تمام حالت‌ها تطابق وجود داشته است. در پایان نیز پیام زیر دیده می‌شود: ControlUnit_tb PASSED بنابراین واحد کنترل برای دستورهای تست‌شده، سیگنال‌های کنترلی صحیح تولید کرده و با ISA تعریف‌شده پردازنده سازگار است.

9-2. تست شمارنده برنامه PC_tb

هدف این تست، بررسی عملکرد ماژول PC به عنوان ثبات نگهدارنده آدرس دستور جاری است. این ماژول باید در زمان فعال بودن reset مقدار خروجی را صفر کند و پس از غیرفعال شدن reset، در هر لبه بالارونده کلاک مقدار next_pc را در خروجی pc_out ثبت کند.

در معماری تک‌سیکله، PC نقش بسیار مهمی دارد؛ زیرا مشخص می‌کند در هر سیکل کدام دستور از حافظه دستور خوانده شود. بنابراین صحت رفتار reset و ثبت مقدار بعدی PC برای اجرای ترتیبی، branch و jump ضروری است.

	Msgs						
/PC_tb/dk	1'h0						
/PC_tb/next_pc	12'h000	000		123	124		
/PC_tb/pc_out	12'h000	000			123		
/PC_tb/reset	1'h1						

تحلیل Waveform

در waveform، سیگنال‌های clk، reset، next_pc و pc_out نمایش داده شده‌اند. در ابتدای شبیه‌سازی، reset فعال است و مقدار pc_out برابر 000 می‌ماند. این رفتار نشان می‌دهد که ماژول PC در حالت reset مقدار آدرس را به ابتدای برنامه برمی‌گرداند.

پس از غیرفعال شدن reset، مقدار next_pc برابر 123 قرار داده شده است. در لبه بالارونده بعدی کلاک، مقدار pc_out نیز به 123 تغییر می‌کند. این موضوع نشان می‌دهد که PC مقدار ورودی را به صورت سنکرون و در لبه کلاک ذخیره می‌کند، نه بلافاصله پس از تغییر next_pc.

در ادامه، مقدار next_pc به 124 تغییر داده شده است. تا قبل از رسیدن لبه بالارونده بعدی، مقدار pc_out همان مقدار قبلی باقی می‌ماند. این نکته نشان می‌دهد که خروجی PC فقط در لبه فعال کلاک به‌روزرسانی می‌شود. در انتهای waveform نیز دوباره با فعال شدن reset، مقدار pc_out به صفر بازمی‌گردد.

این رفتار دقیقاً با نقش PC در پردازنده سازگار است؛ زیرا مقدار بعدی PC توسط منطق branch و jump ساخته می‌شود، اما فقط در لبه کلاک در ثبات PC ذخیره می‌گردد.

```
# ===== PC_tb =====
# PC_tb PASSED
# ** Note: $stop
#      Time: 27 ns  It
```

9-3. تست افزایش‌دهنده PC یا AdderPC_tb

هدف این تست، بررسی ماژول AdderPC است. این ماژول مقدار فعلی PC را دریافت کرده و مقدار $PC + 1$ را تولید می‌کند. خروجی این ماژول برای اجرای ترتیبی برنامه استفاده می‌شود؛ یعنی اگر دستور branch یا jump انجام نشود، پردازنده باید دستور بعدی را از آدرس بعدی حافظه دستور بخواند.

از آنجا که آدرس‌دهی پردازنده 12 بیتی است، این تست علاوه بر حالت‌های عادی، حالت سرریز یا wrap-around را نیز بررسی می‌کند.

	Msgs						
/AdderPC_tb/a	12'hfff	000	001	00f	7fe	fff	
/AdderPC_tb/result	12'h000	001	002	010	7ff	000	

تحليل Waveform

در waveform، ورودی a و خروجی result نمایش داده شده‌اند. در حالت‌های عادی، خروجی دقیقاً یک واحد بیشتر از ورودی است:

a = 000 -> result = 001

a = 001 -> result = 002

a = 00F -> result = 010

$a = 7FE \rightarrow \text{result} = 7FE$

این مقادیر نشان می‌دهند که مازول افزایش‌دهنده برای آدرس‌های معمولی درست عمل می‌کند.

نکته مهم این تست، مقدار ورودی FFF است. چون عرض آدرس 12 بیت است، مقدار بعد از FFF نمی‌تواند 1000 باشد و باید به 000 برگردد. در waveform نیز دیده می‌شود:

a = FFF -> result = 000

این رفتار نشان‌دهنده wrap-around طبیعی جمع 12 بیتی است. بنابراین AdderPC هم در حالت‌های عادی و هم در مرز نهایی آدرس‌دهی عملکرد درستی دارد.

```
# ===== AdderPC_tb =====
# OK: a=000 result=001
# OK: a=001 result=002
# OK: a=00f result=010
# OK: a=7fe result=7ff
# OK: a=fff result=000
# AdderPC tb PASSED
```

9-4. تست واحد محاسباتی و منطقی ALU_tb

هدف این تست، بررسی عملکرد واحد ALU در اجرای عملیات حسابی و منطقی مختلف است. ALU یکی از اصلی ترین بخش های پردازنده است، زیرا عملیات هایی مانند جمع، تفریق، AND، OR، NOT و انتقال داده توسط آن انجام می شوند.

در این تست، ورودی‌های a و b همراه با سیگنال کنترلی alu_op تغییر داده شده‌اند و خروجی result و سیگنال zero_flag بررسی شده‌اند. سیگنال zero_flag اهمیت ویژه‌ای دارد، زیرا در دستور BranchZ برای تشخیص برابر بودن R0 و رجیستر مورد نظر استفاده می‌شود.

[illegible]

در waveform، مقدار alu_op از حالت‌های مختلف عبور می‌کند و در هر حالت، خروجی result مطابق عملیات مورد انتظار تغییر می‌کند.

در ابتدای تست، برای $alu_op = 0$ عملیات جمع بررسی شده است. با قرار گرفتن ورودی‌ها به صورت:

$$a = 0003$$

$$b = 0005$$

$$result = 0008$$

این مقدار نشان می‌دهد عملیات جمع به درستی انجام شده است.

در حالت بعدی، $alu_op = 1$ برای تفریق بررسی شده است. برای مثال:

$$a = 0008$$

$$b = 0005$$

$$result = 0003$$

همچنین در حالت دیگری، تفریق دو مقدار برابر باعث تولید خروجی صفر شده است:

$$a = 0005$$

$$b = 0005$$

$$result = 0000$$

در همین لحظه، سیگنال zero_flag فعال می‌شود. این بخش از تست بسیار مهم است، زیرا نشان می‌دهد ALU می‌تواند نتیجه صفر را به درستی تشخیص دهد و این خروجی برای دستورهای branch قابل استفاده است.

در ادامه، عملیات منطقی نیز بررسی شده‌اند. برای نمونه، در حالت AND، ورودی‌های مشابه 00F0 و 0FF0 خروجی 00F0 تولید می‌کنند. در حالت OR نیز ترکیب همین ورودی‌ها خروجی 0FF0 را نتیجه می‌دهد. این رفتار نشان می‌دهد عملیات منطقی پایه به درستی پیاده‌سازی شده‌اند.

در بخش‌های بعدی waveform، حالت‌های دیگر alu_op نیز بررسی شده‌اند؛ از جمله عملیات NOT و عبور مستقیم یکی از ورودی‌ها. برای مثال، مقدارهایی مانند ABCD و 1234 در خروجی دیده می‌شوند که نشان‌دهنده عملکرد درست مسیرهای انتقال داده در ALU است.

در انتهای تست نیز یک حالت صفر دیگر مشاهده می‌شود که دوباره باعث فعال شدن zero_flag شده است. بنابراین ALU علاوه بر تولید نتیجه صحیح عملیات، وضعیت صفر بودن خروجی را نیز درست گزارش می‌کند.

```
# ===== ALU_tb =====
# OK: op=000 a=0003 b=0005 result=0008 zero=0
# OK: op=001 a=0008 b=0005 result=0003 zero=0
# OK: op=001 a=0005 b=0005 result=0000 zero=1
# OK: op=010 a=00f0 b=0ff0 result=00f0 zero=0
# OK: op=011 a=00f0 b=0f0f result=0fff zero=0
# OK: op=100 a=00f0 b=1234 result=ff0f zero=0
# OK: op=101 a=abcd b=1234 result=abcd zero=0
# OK: op=110 a=abcd b=1234 result=1234 zero=0
# OK: op=111 a=1111 b=2222 result=0000 zero=1
# ALU_tb PASSED
```

9-5. تست حافظه دستور InstructionMemory_tb

هدف این تست، بررسی صحت عملکرد حافظه دستور و اطمینان از بارگذاری صحیح فایل program.hex است. ماژول InstructionMemory با دریافت یک آدرس ۱۲ بیتی، دستور ۱۶ بیتی متناظر با همان آدرس را در خروجی قرار می‌دهد.

از آنجا که دستورها از فایل خارجی program.hex و با استفاده از دستور readmemh بارگذاری می‌شوند، این تست علاوه بر بررسی عملکرد حافظه دستور، صحت مسیر فایل برنامه و بارگذاری صحیح آن را نیز تأیید می‌کند.

	Msgs												
/InstructionMemory_tb/address	12'h100	000	001	002	003	004	005	006	007	008	009	00a	00b
/InstructionMemory_tb/instruction	16'h8080	c005	8201	c003	8401	8202	8404	8208	8601	420c	8210	480c	c001

00c	00d	00e	00f	010	011	012	013	014	015	016	017	018	019	01a	01b	01c	020	100
8220	8240	e00f	f0f0	d00f	1020	cfff	0020	8a01	4618	4a19	c001	f800	8080	201c	c123	201c	8080	

تحلیل Waveform

در waveform مربوط به InstructionMemory_tb، سیگنال address آدرس‌های مختلف برنامه را دریافت می‌کند و خروجی instruction مقدار دستور متناظر با همان آدرس را نشان می‌دهد. برای مثال در آدرس‌های ابتدایی برنامه مقادیر زیر مشاهده می‌شوند:

address = 000 -> instruction = C005

address = 001 -> instruction = 8201

address = 002 -> instruction = C003

address = 003 -> instruction = 8401

این مقادیر با محتوای فایل program.hex منطبق هستند و نشان می‌دهند برنامه به درستی در حافظه دستور بارگذاری شده است.

در ادامه، آدرس‌های مهم دیگری نیز بررسی شده‌اند. برای مثال در آدرس 010 مقدار زیر دیده می‌شود:

address = 010 -> instruction = D00F

این دستور همان Subi 00F است که در برنامه نهایی برای رسیدن به مقدار نهایی 00EB استفاده شده است.

همچنین در آدرس 01C مقدار زیر مشاهده می‌شود:

address = 01C -> instruction = 201C

این دستور، پرش به آدرس خودش است و باعث ایجاد self-loop در انتهای برنامه می‌شود. در تست نهایی پردازنده نیز انتظار می‌رود مقدار PC در همین آدرس باقی بماند.

در آدرس‌های خارج از محدوده اصلی برنامه، مانند 020 و 100، مقدار 8080 دیده می‌شود که به عنوان دستور امن یا NOP در نظر گرفته شده است.

```
# ===== InstructionMemory_tb =====
# OK: addr=000 instruction=c005
# OK: addr=001 instruction=8201
# OK: addr=002 instruction=c003
# OK: addr=003 instruction=8401
# OK: addr=004 instruction=8202
# OK: addr=005 instruction=8404
# OK: addr=006 instruction=8208
# OK: addr=007 instruction=8601
# OK: addr=008 instruction=420c
# OK: addr=009 instruction=8210
# OK: addr=00a instruction=480c
# OK: addr=00b instruction=c001
# OK: addr=00c instruction=8220
# OK: addr=00d instruction=8240
# OK: addr=00e instruction=e00f
# OK: addr=00f instruction=f0f0
# OK: addr=010 instruction=d00f
# OK: addr=011 instruction=l020
# OK: addr=012 instruction=cfff
# OK: addr=013 instruction=0020
# OK: addr=014 instruction=8a01
# OK: addr=015 instruction=4618
# OK: addr=016 instruction=4a19
# OK: addr=017 instruction=c001
# OK: addr=018 instruction=f800
# OK: addr=019 instruction=8080
# OK: addr=01a instruction=201c
# OK: addr=01b instruction=c123
# OK: addr=01c instruction=201c
# OK: addr=020 instruction=8080
# OK: addr=100 instruction=8080
# InstructionMemory_tb PASSED
```

تحلیل Transcript

در خروجی transcript، برای هر آدرس بررسی شده، مقدار دستور خوانده شده با مقدار مورد انتظار مقایسه شده و پیام OK چاپ شده است. برای نمونه:

OK: addr=010 instruction=d00f

OK: addr=01c instruction=201c

OK: addr=020 instruction=8080

OK: addr=100 instruction=8080

وجود پیام نهایی زیر نشان می‌دهد که تمام دستورهای خوانده شده از حافظه دستور با مقادیر مورد انتظار تست بنج منطبق بوده‌اند:

InstructionMemory_tb PASSED بنابراین تست تأیید می‌کند که فایل program.hex به درستی بارگذاری شده و حافظه دستور برای اجرای برنامه نهایی آماده است.

9-6. تست واحد گسترش (immediate) ExtendUnit_tb

هدف این تست، بررسی عملکرد ماژول ExtendUnit در تبدیل مقدار immediate دوازدهبیتی به مقدار شانزدهبیتی است. این ماژول در دستورهای immediate استفاده می‌شود و بسته به نوع دستور، باید یکی از دو حالت sign-extension یا zero-extension را انجام دهد.

در دستورهای حسابی immediate مانند Addi و Subi، مقدار immediate باید به صورت علامت دار تفسیر شود؛ بنابراین sign-extension لازم است. اما در دستورهای منطقی immediate مانند Andi و Ori، مقدار immediate به صورت بدون علامت در نظر گرفته می‌شود و zero-extension مناسب‌تر است.

	Msgs								
/ExtendUnit_tb/ExtSel	1'h 1								
/ExtendUnit_tb/in	12'h 123	123	7ff	800	fff	123	800	fff	
/ExtendUnit_tb/out	16'h 0123	0123	07ff	f800	ffff	0123	0800	0fff	

تحلیل Waveform

در waveform، سه سیگنال in، ExtSel و out بررسی شده‌اند. ابتدا مقدار ExtSel در حالت sign-extension قرار دارد. در این حالت، اگر بیت بالایی ورودی دوازدهبیتی صفر باشد، بیت‌های بالایی خروجی شانزدهبیتی نیز صفر می‌شوند. برای مثال:

in = 12'h123 -> out = 16'h0123

in = 12'h7FF -> out = 16'h07FF

اما زمانی که بیت علامت ورودی برابر 1 باشد، خروجی با تکرار بیت علامت در بیت‌های بالایی ساخته می‌شود. برای نمونه:

in = 12'h800 -> out = 16'hF800

in = 12'hFFF -> out = 16'hFFFF

این قسمت نشان می‌دهد که sign-extension درست انجام شده است.

در بخش دوم waveform، مقدار ExtSel تغییر می‌کند و ماژول وارد حالت zero-extension می‌شود. در این حالت، بدون توجه به بیت علامت ورودی، بیت‌های بالایی خروجی با صفر پر می‌شوند. بنابراین:

in = 12'h800 -> out = 16'h0800

in = 12'hFFF -> out = 16'h0FFF

این رفتار برای دستورهای منطقی immediate ضروری است، زیرا در این دستورها immediate نباید به صورت عدد منفی تفسیر شود.

```
# ===== ExtendUnit_tb =====
# OK: in=123 ExtSel=0 out=0123
# OK: in=7ff ExtSel=0 out=07ff
# OK: in=800 ExtSel=0 out=f800
# OK: in=fff ExtSel=0 out=ffff
# OK: in=123 ExtSel=1 out=0123
# OK: in=800 ExtSel=1 out=0800
# OK: in=fff ExtSel=1 out=0fff
# ExtendUnit_tb PASSED
```

9-7. تست حافظه داده DataMemory_tb

هدف و روند تست

هدف این تست، بررسی عملکرد ماژول DataMemory در عملیات خواندن و نوشتن است. این ماژول در دستورهای Load و Store استفاده می‌شود. در دستور Store، مقدار موجود در R0 باید در حافظه داده نوشته شود و در دستور Load، مقدار حافظه باید خوانده شده و در مسیر write-back قرار گیرد.

در این طراحی، نوشتن در حافظه به صورت سنکرون و در لبه کلاک انجام می شود، اما خواندن از حافظه به صورت ترکیبی و وابسته به فعال بودن MemRead است.

	Mops																
/DataMemory_tb/address	12h011	000	010							011							
/DataMemory_tb/clock	1'h1																
/DataMemory_tb/MemRead	1'h0																
/DataMemory_tb/MemWrite	1'h1																
/DataMemory_tb/read_data	16'h0000	0000							1234	0000							
/DataMemory_tb/write_data	16'habcd	0000		1234						abcd							

تحليل Waveform

در waveform، سیگنال‌های address، clk، MemRead، MemWrite، write_data و read_data نمایش داده شده‌اند.

در ابتدای تست، مقدار آدرس برابر 000 قرار دارد و خروجی `read_data` مقدار صفر را نشان می‌دهد. سپس آدرس به 010 تغییر داده می‌شود و مقدار `write_data` برابر 1234 قرار می‌گیرد. با فعال شدن `MemWrite` و رسیدن لبه کلاک، مقدار 1234 در آدرس 010 نوشته می‌شود.

پس از آن، با فعال شدن MemRead، مقدار ذخیره شده در همان آدرس روی خروجی read_data ظاهر می‌شود:

address = 010

```
read_data = 1234
```

این بخش نشان می‌دهد که عملیات نوشتن و سپس خواندن از همان خانه حافظه به درستی انجام شده است.

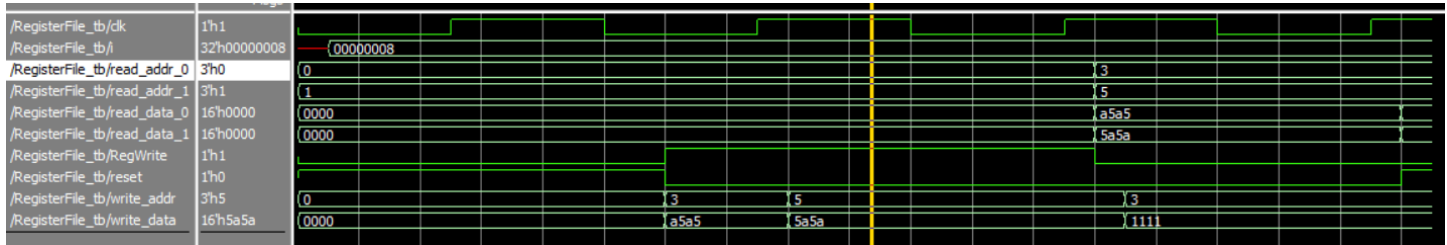
در ادامه تست، آدرس به 011 تغییر می‌کند و مقدار write_data برابر ABCD قرار می‌گیرد. با فعال بودن MemWrite و اعمال کلاک، این مقدار نیز در حافظه نوشته می‌شود. زمانی که خواندن فعال نیست، خروجی read_data صفر نمایش داده می‌شود؛ این رفتار نشان می‌دهد خروجی خواندن به سیگنال MemRead وابسته است.

این تست نشان می‌دهد که حافظه داده برای اجرای دستورهای Load و Store آماده است: داده در لبه کلاک نوشته می‌شود و هنگام فعال شدن خواندن، مقدار ذخیره‌شده از آدرس انتخاب‌شده برگردانده می‌شود.

```
# ===== DataMemory_tb :
# DataMemory_tb PASSED
# ** Note: $stop      : 1
# Time: 16.000000000000000
```

9-8. تست فایل ثبات‌ها RegisterFile_tb

هدف این تست، بررسی عملکرد ماژول RegisterFile در سه حالت اصلی reset، نوشتن و خواندن است. فایل ثبات‌ها شامل ۸ رجیستر ۱۶ بیتی است که با آدرس‌های ۳ بیتی انتخاب می‌شوند. این ماژول دارای دو پورت خواندن و یک پورت نوشتن است؛ بنابراین امکان خواندن هم‌زمان از دو رجیستر و نوشتن در یک رجیستر در لبه کلاک وجود دارد.



تحلیل Waveform

در ابتدای شبیه‌سازی، سیگنال reset فعال است و خروجی‌های خواندن read_data_0 و read_data_1 مقدار 0000 را نشان می‌دهند. این موضوع نشان می‌دهد رجیسترها به درستی مقداردهی اولیه شده‌اند.

در ادامه، با فعال شدن RegWrite، مقدارهای A5A5 و 5A5A به ترتیب در رجیسترهای شماره 3 و 5 نوشته می‌شوند. سپس آدرس‌های خواندن روی مقدارهای زیر قرار می‌گیرند:

read_addr_0 = 3

read_addr_1 = 5

و خروجی‌ها به شکل زیر مشاهده می‌شوند:

read_data_0 = A5A5

read_data_1 = 5A5A

بنابراین داده‌های نوشته‌شده به درستی در رجیسترها ذخیره شده و از طریق پورت‌های خواندن قابل مشاهده هستند. همچنین در ادامه تست، مقدار جدیدی مانند 1111 روی رجیستر 3 نوشته می‌شود تا قابلیت بازنویسی مقدار یک رجیستر نیز بررسی شود.

در نتیجه، این تست صحت سه عملکرد اصلی فایل ثبات‌ها را تأیید می‌کند:

1. صفر شدن رجیسترها هنگام reset

2. نوشتن سنکرون داده در رجیستر مقصد هنگام فعال بودن RegWrite

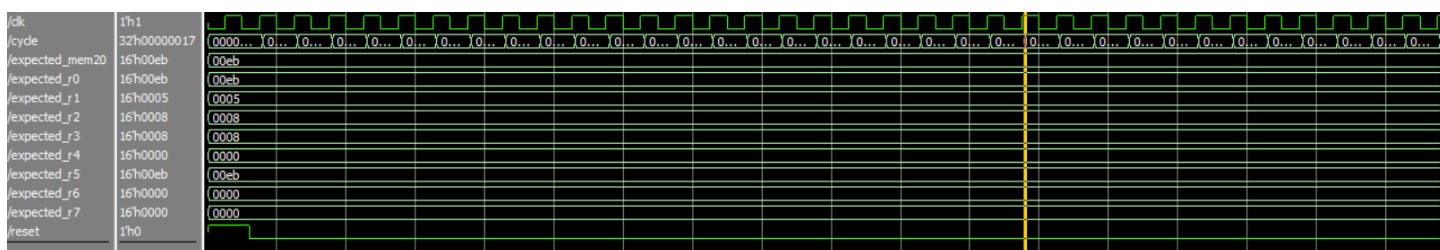
3. خواندن هم‌زمان و صحیح از دو رجیستر انتخاب‌شده

```
# ===== RegisterFile_tb =====  
# RegisterFile_tb PASSED
```

9.9. تحلیل تست نهایی پردازنده SingleCycleProcessor_tb

تست بنچ SingleCycleProcessor_tb مهم‌ترین مرحله‌ی اعتبارسنجی طراحی است، زیرا در این تست دیگر عملکرد یک ماژول به صورت جداگانه بررسی نمی‌شود، بلکه کل پردازنده‌ی Single-Cycle به صورت یک سیستم یکپارچه تحت آزمایش قرار می‌گیرد. در این مرحله، ماژول‌های اصلی شامل شمارنده برنامه PC، حافظه دستور، واحد کنترل، فایل ثبات‌ها، واحد توسعه مقدار فوری، ALU، حافظه داده و مسیرهای انتخاب داده همگی به یکدیگر متصل شده‌اند و برنامه‌ی ذخیره‌شده در فایل program.hex به صورت کامل اجرا می‌شود.

هدف اصلی این تست آن است که مشخص شود پردازنده می‌تواند پس از ریست، دستورات برنامه را به ترتیب صحیح دریافت و اجرا کند، مقادیر رجیسترها و حافظه را مطابق انتظار تغییر دهد و در نهایت در آدرس پایان برنامه وارد حلقه‌ی پایدار شود. نتیجه‌ی این تست با مقایسه‌ی مقادیر واقعی پردازنده با مقادیر مورد انتظار در انتهای شبیه‌سازی بررسی می‌شود.



سیگنال‌های موجود در Waveform

در waveform این تست بنچ، سیگنال‌های اصلی زیر مشاهده می‌شوند:

سیگنال	توضیح
clk	کلاک اصلی پردازنده که اجرای سیکل به سیکل برنامه با آن انجام می‌شود.
reset	سیگنال مقداردهی اولیه پردازنده. در ابتدای شبیه‌سازی فعال است و سپس غیرفعال می‌شود.
cycle	شمارنده سیکل‌های شبیه‌سازی که برای ردیابی روند اجرای برنامه استفاده شده است.
expected_r0 تا expected_r7	مقادیر مورد انتظار رجیسترهای عمومی در انتهای اجرای برنامه.
expected_mem20	مقدار مورد انتظار خانه حافظه داده با آدرس 0x0200 در پایان تست.

در این waveform تمرکز اصلی روی کنترل روند اجرای کلی تست است، نه نمایش تمام سیگنال‌های داخلی datapath. به همین دلیل، سیگنال‌های مورد انتظار نهایی در کنار clk، reset و cycle قرار داده شده‌اند تا مشخص باشد تست بنچ در پایان شبیه‌سازی چه مقادیری را به عنوان خروجی صحیح پردازنده در نظر می‌گیرد.

در تصویر waveform مشاهده می شود که پس از شروع شبیه سازی، سیگنال reset ابتدا فعال بوده و سپس صفر می شود. پس از غیرفعال شدن reset، پردازنده وارد مرحله اجرای برنامه می شود و مقدار cycle با هر سیکل کلاک افزایش می یابد. ثابت بودن سیگنال های *_expected در طول شبیه سازی نشان می دهد که این مقادیر صرفاً نقش مرجع نهایی را دارند و در انتهای تست برای مقایسه با خروجی واقعی پردازنده استفاده می شوند.

```
# cycle=1 pc=001 instr=8201 R0=0005 R1=0000 R2=0000 R3=0000 R4=0000 R5=0000 R6=0000 R7=0000 MEM20=0000
# cycle=2 pc=002 instr=c003 R0=0005 R1=0005 R2=0000 R3=0000 R4=0000 R5=0000 R6=0000 R7=0000 MEM20=0000
# cycle=3 pc=003 instr=8401 R0=0008 R1=0005 R2=0000 R3=0000 R4=0000 R5=0000 R6=0000 R7=0000 MEM20=0000
# cycle=4 pc=004 instr=8202 R0=0008 R1=0005 R2=0008 R3=0000 R4=0000 R5=0000 R6=0000 R7=0000 MEM20=0000
# cycle=5 pc=005 instr=8404 R0=0005 R1=0005 R2=0008 R3=0000 R4=0000 R5=0000 R6=0000 R7=0000 MEM20=0000
# cycle=6 pc=006 instr=8208 R0=000d R1=0005 R2=0008 R3=0000 R4=0000 R5=0000 R6=0000 R7=0000 MEM20=0000
# cycle=7 pc=007 instr=8601 R0=0008 R1=0005 R2=0008 R3=0000 R4=0000 R5=0000 R6=0000 R7=0000 MEM20=0000
# cycle=8 pc=008 instr=420c R0=0008 R1=0005 R2=0008 R3=0008 R4=0000 R5=0000 R6=0000 R7=0000 MEM20=0000
# cycle=9 pc=009 instr=8210 R0=0008 R1=0005 R2=0008 R3=0008 R4=0000 R5=0000 R6=0000 R7=0000 MEM20=0000
# cycle=10 pc=00a instr=480c R0=0000 R1=0005 R2=0008 R3=0008 R4=0000 R5=0000 R6=0000 R7=0000 MEM20=0000
# cycle=11 pc=00c instr=8220 R0=0000 R1=0005 R2=0008 R3=0008 R4=0000 R5=0000 R6=0000 R7=0000 MEM20=0000
# cycle=12 pc=00d instr=8240 R0=0005 R1=0005 R2=0008 R3=0008 R4=0000 R5=0000 R6=0000 R7=0000 MEM20=0000
# cycle=13 pc=00e instr=e00f R0=fffa R1=0005 R2=0008 R3=0008 R4=0000 R5=0000 R6=0000 R7=0000 MEM20=0000
# cycle=14 pc=00f instr=f0f0 R0=000a R1=0005 R2=0008 R3=0008 R4=0000 R5=0000 R6=0000 R7=0000 MEM20=0000
# cycle=15 pc=010 instr=d00f R0=00fa R1=0005 R2=0008 R3=0008 R4=0000 R5=0000 R6=0000 R7=0000 MEM20=0000
# cycle=16 pc=011 instr=1020 R0=00eb R1=0005 R2=0008 R3=0008 R4=0000 R5=0000 R6=0000 R7=0000 MEM20=0000
# cycle=17 pc=012 instr=cfff R0=00eb R1=0005 R2=0008 R3=0008 R4=0000 R5=0000 R6=0000 R7=0000 MEM20=00eb
# cycle=18 pc=013 instr=0020 R0=00ea R1=0005 R2=0008 R3=0008 R4=0000 R5=0000 R6=0000 R7=0000 MEM20=00eb
# cycle=19 pc=014 instr=8a01 R0=00eb R1=0005 R2=0008 R3=0008 R4=0000 R5=0000 R6=0000 R7=0000 MEM20=00eb
# cycle=20 pc=015 instr=4618 R0=00eb R1=0005 R2=0008 R3=0008 R4=0000 R5=00eb R6=0000 R7=0000 MEM20=00eb
# cycle=21 pc=016 instr=4a19 R0=00eb R1=0005 R2=0008 R3=0008 R4=0000 R5=00eb R6=0000 R7=0000 MEM20=00eb
# cycle=22 pc=019 instr=8080 R0=00eb R1=0005 R2=0008 R3=0008 R4=0000 R5=00eb R6=0000 R7=0000 MEM20=00eb
# cycle=23 pc=01a instr=201c R0=00eb R1=0005 R2=0008 R3=0008 R4=0000 R5=00eb R6=0000 R7=0000 MEM20=00eb
# cycle=24 pc=01c instr=201c R0=00eb R1=0005 R2=0008 R3=0008 R4=0000 R5=00eb R6=0000 R7=0000 MEM20=00eb
# cycle=25 pc=01c instr=201c R0=00eb R1=0005 R2=0008 R3=0008 R4=0000 R5=00eb R6=0000 R7=0000 MEM20=00eb
# cycle=26 pc=01c instr=201c R0=00eb R1=0005 R2=0008 R3=0008 R4=0000 R5=00eb R6=0000 R7=0000 MEM20=00eb
# cycle=27 pc=01c instr=201c R0=00eb R1=0005 R2=0008 R3=0008 R4=0000 R5=00eb R6=0000 R7=0000 MEM20=00eb
# cycle=28 pc=01c instr=201c R0=00eb R1=0005 R2=0008 R3=0008 R4=0000 R5=00eb R6=0000 R7=0000 MEM20=00eb
# cycle=29 pc=01c instr=201c R0=00eb R1=0005 R2=0008 R3=0008 R4=0000 R5=00eb R6=0000 R7=0000 MEM20=00eb
# cycle=30 pc=01c instr=201c R0=00eb R1=0005 R2=0008 R3=0008 R4=0000 R5=00eb R6=0000 R7=0000 MEM20=00eb
# cycle=31 pc=01c instr=201c R0=00eb R1=0005 R2=0008 R3=0008 R4=0000 R5=00eb R6=0000 R7=0000 MEM20=00eb
# cycle=32 pc=01c instr=201c R0=00eb R1=0005 R2=0008 R3=0008 R4=0000 R5=00eb R6=0000 R7=0000 MEM20=00eb
# cycle=33 pc=01c instr=201c R0=00eb R1=0005 R2=0008 R3=0008 R4=0000 R5=00eb R6=0000 R7=0000 MEM20=00eb
# cycle=34 pc=01c instr=201c R0=00eb R1=0005 R2=0008 R3=0008 R4=0000 R5=00eb R6=0000 R7=0000 MEM20=00eb
# cycle=35 pc=01c instr=201c R0=00eb R1=0005 R2=0008 R3=0008 R4=0000 R5=00eb R6=0000 R7=0000 MEM20=00eb
```

روند اجرای برنامه در Transcript

جزئیات واقعی اجرای برنامه در transcript شبیه سازی قابل مشاهده است. در هر سیکل، مقدار PC، دستور جاری، مقدار رجیستری های R0 تا R7 و مقدار حافظه MEM[020] چاپ شده است. این ساختار باعث می شود بتوان روند اجرای برنامه را به صورت مرحله به مرحله دنبال کرد و تغییرات ناشی از هر دستور را مشاهده نمود.

در سیکل های ابتدایی، پردازنده پس از خروج از حالت reset شروع به اجرای دستورهای برنامه می کند. در این بخش، مقادیر اولیه در رجیسترها قرار داده می شوند و سپس تعدادی عملیات محاسباتی و انتقال داده انجام می گیرد. برای مثال، در سیکل های اولیه مقدارهای 0005 و 0008 در رجیستری های مربوطه ظاهر می شوند و در ادامه این مقادیر در عملیات بعدی مورد استفاده قرار می گیرند.

بخشی از روند اجرای اولیه در transcript به صورت زیر دیده می شود:

cycle=1 pc=001 instr=8201 R0=0005 ...

cycle=2 pc=002 instr=C003 R1=0005 ...

cycle=3 pc=003 instr=8401 R0=0008 R2=0008 ...

cycle=4 pc=004 instr=8202 R0=0005 R2=0008 ...

این بخش نشان می‌دهد که پردازنده پس از شروع اجرا، به درستی مقدار PC را افزایش داده و دستورهای پشت سر هم را از حافظه دستور دریافت می‌کند. همچنین تغییر مقادیر رجیسترها در هر سیکل نشان می‌دهد مسیرهای نوشتن در Register File، تولید سیگنال‌های کنترل و عملیات ALU به درستی با یکدیگر هماهنگ شده‌اند.

در ادامه، دستورهای مختلف برنامه باعث تغییر مقدار رجیستر R0 و آماده‌سازی مقدار نهایی می‌شوند. طبق transcript، مقدار R0 در چند مرحله تغییر می‌کند و در نهایت به مقدار نهایی 00EB می‌رسد:

cycle=13 pc=00e instr=E00F R0=FFFA ...

cycle=14 pc=00f instr=F0F0 R0=000A ...

cycle=15 pc=010 instr=D00F R0=00FA ...

cycle=16 pc=011 instr=1020 R0=00EB ...

این قسمت اهمیت زیادی دارد، زیرا نشان می‌دهد ALU و واحد کنترل توانسته‌اند دستورهای متوالی مربوط به عملیات محاسباتی و منطقی را درست تفسیر و اجرا کنند. تغییرات مرحله‌به‌مرحله‌ی R0 در transcript تأیید می‌کند که نتیجه عملیات فقط در انتهای برنامه به درستی ظاهر نشده، بلکه مسیر اجرای آن نیز مطابق انتظار طی شده است.

بررسی حافظه داده و انتقال مقدار نهایی

پس از محاسبه مقدار نهایی 00EB، این مقدار در حافظه داده ذخیره می‌شود. در transcript مشاهده می‌شود که مقدار MEM[020] ابتدا برابر 0000 است و سپس در سیکل مربوط به عملیات حافظه به مقدار 00EB تغییر می‌کند:

cycle=17 pc=012 instr=CFFF R0=00EB ... MEM20=0000

cycle=18 pc=013 instr=0020 R0=00EA ... MEM20=00EB

تغییر مقدار MEM20 از 0000 به 00EB نشان می‌دهد که مسیر نوشتن در حافظه داده، سیگنال کنترلی مربوط به نوشتن حافظه و آدرس‌دهی حافظه درست عمل کرده‌اند. این بخش از تست برای پردازنده بسیار مهم است، زیرا صحت عملکرد حافظه داده فقط به ALU وابسته نیست، بلکه نیازمند هماهنگی بین واحد کنترل، مسیر آدرس، داده ورودی حافظه و سیگنال MemWrite است.

در ادامه، مقدار نهایی از حافظه یا مسیر محاسباتی مربوطه به رجیستر R5 منتقل می‌شود. طبق transcript، مقدار R5 در ابتدا صفر است و سپس به مقدار 00EB تغییر می‌کند:

cycle=19 pc=014 instr=8A01 ... R5=0000 MEM20=00EB

cycle=20 pc=015 instr=4618 ... R5=00EB MEM20=00EB

این نتیجه نشان می‌دهد که علاوه بر ذخیره‌سازی مقدار در حافظه، مسیر انتقال داده به رجیستر مقصد نیز درست عمل کرده است. در نتیجه، هم بخش حافظه داده و هم بخش نوشتن نتیجه در Register File در تست نهایی تأیید می‌شوند.

رسیدن پردازنده به حالت پایدار نهایی

در بخش پایانی اجرای برنامه، پردازنده به دستور پرش نهایی می‌رسد. در transcript مشاهده می‌شود که پس از اجرای دستورهای پایانی، مقدار PC روی آدرس 01C ثابت می‌شود و دستور موجود در این آدرس برابر 201C است:

cycle=24 pc=01c instr=201c ...

cycle=25 pc=01c instr=201c ...

cycle=26 pc=01c instr=201c ...

...

cycle=35 pc=01c instr=201c ...

ثابت ماندن PC روی مقدار 01C نشان می‌دهد که پردازنده وارد یک self-loop شده است. دستور 201C به‌عنوان دستور پرش به آدرس 0x01C عمل می‌کند و باعث می‌شود پردازنده در انتهای برنامه در همان آدرس باقی بماند. این رفتار برای پایان کنترل‌شده‌ی برنامه در تست‌بنچ استفاده شده است؛ یعنی به‌جای ادامه اجرای آدرس‌های نامعتبر یا تصادفی، پردازنده در یک حالت پایدار قابل بررسی متوقف می‌شود.

بنابراین، رسیدن به وضعیت زیر نشانه پایان صحیح اجرای برنامه است:

Final PC = 01C

Final instruction = 201C

بررسی نهایی مقادیر رجیسترها و حافظه

پس از رسیدن پردازنده به حلقه پایانی، تست‌بنچ مقادیر واقعی رجیسترها و حافظه را با مقادیر مورد انتظار مقایسه می‌کند. بخش Final checks در transcript نشان می‌دهد که تمام مقادیر بررسی شده صحیح هستند:

```

# -----
# Final checks
# -----
# OK: R0 = 00eb
# OK: R1 = 0005
# OK: R2 = 0008
# OK: R3 = 0008
# OK: R4 = 0000
# OK: R5 = 00eb
# OK: R6 = 0000
# OK: R7 = 0000
# OK: MEM[0x020] = 00eb
# OK: Final PC = 01c (self-loop reached)
# OK: Final instruction = 201c (Jump 0x01C)
# -----
# TEST PASSED
# -----

```

این خروجی نشان می‌دهد که تمام رجیسترهای عمومی پردازنده مقدار صحیح نهایی را دارند. همچنین مقدار خانه حافظه MEM[0x020] برابر 00EB شده است که با مقدار مورد انتظار تست‌بنج کاملاً تطابق دارد. علاوه بر آن، مقدار نهایی PC و دستور نهایی نیز تأیید می‌کنند که برنامه به نقطه پایان صحیح رسیده و در حلقه پایانی پایدار شده است.

در نهایت، پیام زیر در transcript چاپ شده است: TEST PASSED

جمع‌بندی تست SingleCycleProcessor_tb

نتایج waveform و transcript نشان می‌دهند که پردازنده Single-Cycle طراحی شده، برنامه‌ی موجود در program.hex را به‌درستی اجرا کرده است. سیگنال reset پردازنده را در ابتدای شبیه‌سازی به حالت مشخص برده، سپس با فعال شدن روند عادی اجرا، مقدار PC در هر سیکل به‌درستی به‌روزرسانی شده و دستورهای برنامه از حافظه دستور خوانده شده‌اند.

در طول اجرای برنامه، رجیسترها مطابق انتظار مقداردهی و به‌روزرسانی شده‌اند، عملیات ALU نتیجه صحیح تولید کرده، حافظه داده مقدار نهایی را در آدرس مورد نظر ذخیره کرده و در پایان، پردازنده با رسیدن به دستور Jump 0x01C وارد self-loop پایانی شده است. مقایسه نهایی نیز نشان داده که مقادیر R0 تا R7، مقدار MEM[0x020]، مقدار نهایی PC و دستور نهایی همگی با مقادیر مورد انتظار برابر هستند.

بنابراین، تست SingleCycleProcessor_tb صحت عملکرد کل پردازنده را در سطح سیستم تأیید می‌کند و نشان می‌دهد که تمام واحدهای طراحی شده به‌درستی در datapath یکپارچه شده‌اند. پیام TEST PASSED نتیجه نهایی این اعتبارسنجی است و موفقیت کامل تست نهایی پردازنده را نشان می‌دهد.